

# Relatório Projeto Final

## Integration Gateway

---

Gerenciamento Avançado de Containers

Allef Oliveira Ramos  
Fernanda de Oliveira da Costa  
Pedro Henrique Oliveira Dias  
Juliana Ballin Lima  
Camila Felix dos Reis

Projeto 5 | Turma 2026  
06 de May de 2026

## 1. Contexto do Problema

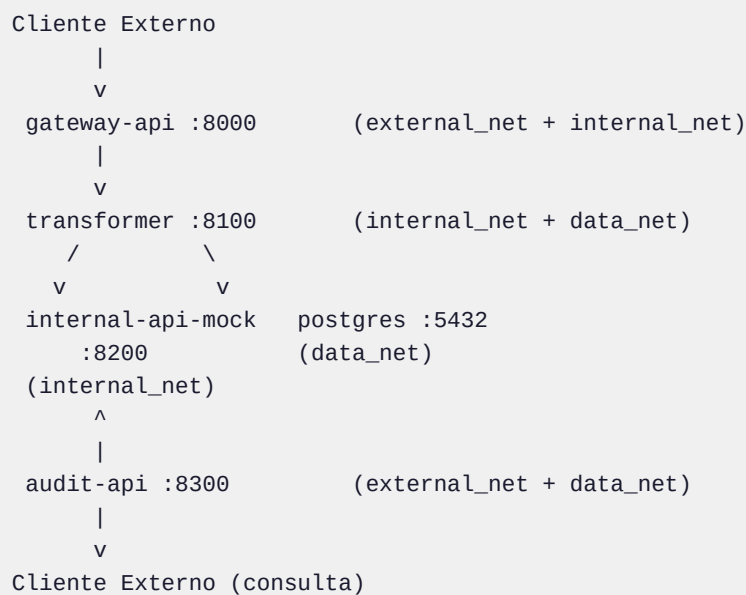
Empresas com sistemas legados recebem dados externos em formatos incompatíveis com seus contratos internos. O projeto implementa uma camada intermediária que atua como ponte entre clientes externos e a API interna, garantindo validação, transformação e rastreabilidade completa das integrações.

### Responsabilidades da camada intermediária

- Recebe pedidos externos via HTTP
- Valida e transforma o payload para o contrato legado
- Encaminha para a API interna
- Registra auditoria completa no banco de dados
- Expõe endpoints de consulta de rastreabilidade

## 2. Arquitetura Geral

O ambiente é composto por cinco serviços orquestrados via Docker Compose, distribuídos em três redes isoladas conforme responsabilidade.



## 3. Serviços

| Serviço     | Imagem      | Porta Host | Função                      |
|-------------|-------------|------------|-----------------------------|
| gateway-api | build local | 8000       | Ponto de entrada externo    |
| transformer | build local | interno    | Valida, transforma e audita |

| Serviço           | Imagem             | Porta Host | Função                    |
|-------------------|--------------------|------------|---------------------------|
| internal-api-mock | build local        | interno    | Simula o sistema legado   |
| postgres          | postgres:15-alpine | nenhuma    | Persistência de auditoria |
| audit-api         | build local        | 8300       | Consulta de histórico     |

*Postgres sem porta exposta: acessível apenas via data\_net.*

## 4. Redes e Isolamento

| Rede         | Serviços                                    | Finalidade                         |
|--------------|---------------------------------------------|------------------------------------|
| external_net | gateway-api, audit-api                      | Tráfego externo controlado         |
| internal_net | gateway-api, transformer, internal-api-mock | Comunicação interna entre serviços |
| data_net     | transformer, audit-api, postgres            | Banco de dados isolado             |

### Por que o gateway precisa de duas redes?

O gateway-api é a fronteira do sistema: recebe requisições de fora via external\_net e as repassa internamente via internal\_net. Sem participar das duas redes, o serviço não conseguiria fazer a ponte entre os dois domínios.

### Por que o Postgres não está na external\_net?

Dados sensíveis de auditoria não devem ser expostos à rede externa. Apenas transformer e audit-api acessam o banco via data\_net, seguindo o princípio do menor privilégio.

## 5. Volumes e Persistência

| Volume        | Tipo         | Finalidade                                |
|---------------|--------------|-------------------------------------------|
| postgres_data | named volume | Auditoria persiste entre reinicializações |
| ./db/init.sql | bind mount   | Script de inicialização do banco          |

## 6. Healthchecks

Todos os serviços expõem /health monitorado pelo Docker. A configuração depends\_on com condition: service\_healthy garante a ordem de inicialização: o transformer só sobe após o postgres estar acessível.

| Serviço           | Endpoint    | Intervalo | Timeout | Retries |
|-------------------|-------------|-----------|---------|---------|
| gateway-api       | GET /health | 10s       | 5s      | 3       |
| transformer       | GET /health | 10s       | 5s      | 3       |
| internal-api-mock | GET /health | 10s       | 5s      | 3       |

| Serviço   | Endpoint    | Intervalo | Timeout | Retries |
|-----------|-------------|-----------|---------|---------|
| audit-api | GET /health | 10s       | 5s      | 3       |
| postgres  | pg_isready  | 10s       | 5s      | 5       |

*O transformer verifica a conexão com o banco em seu /health e só reporta healthy quando o postgres está acessível.*

## 7. Limites de Recursos

Configurados via `deploy.resources.limits` no Compose, garantindo que nenhum serviço consuma recursos além do necessário.

| Serviço           | CPUs | Memória |
|-------------------|------|---------|
| gateway-api       | 0.25 | 128m    |
| transformer       | 0.50 | 256m    |
| internal-api-mock | 0.25 | 128m    |
| audit-api         | 0.25 | 128m    |
| postgres          | 0.50 | 256m    |

## 8. Hardening (Segurança)

As configurações de hardening foram aplicadas nos quatro serviços Python, seguindo o princípio do menor privilégio.

| Configuração | Valor                  | Finalidade                         |
|--------------|------------------------|------------------------------------|
| user         | 1000:1000              | Não roda como root                 |
| read_only    | true                   | Filesystem somente leitura         |
| tmpfs        | /tmp                   | Escrita permitida apenas em /tmp   |
| cap_drop     | ALL                    | Remove todas as capabilities Linux |
| security_opt | no-new-privileges:true | Impede escalada de privilégios     |

### Evidência pelo docker inspect

```
Memory: 268435456   (diferente de 0)
NanoCpus: 500000000 (diferente de 0)
ReadonlyRootfs: true
CapDrop: ["ALL"]
SecurityOpt: ["no-new-privileges:true"]
```

*Evidência 06: docker inspect confirmando hardening*

```
User=1000:1000 Memory=268435456 NanoCpus=500000000 ReadonlyRootfs=true CapDrop=["ALL"] SecurityOpt=["no-new-privileges:true"]
integration-gateway git:(develop) []
```

## 9. Demo: Fluxo Normal

### 9.1 Subir o ambiente

```
docker compose up -d --build
docker compose ps # todos healthy
```

*Evidência 01: docker compose ps com todos os serviços healthy*

```
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED   STATUS    PORTS
integration-gateway-audit-api-1      integration-gateway-audit-api      "python app.py"                       audit-api   22 hours ago    Up 2 hours (healthy)    0.0.0.0:8300->8300/tcp, [::]:8300->8300/tcp
integration-gateway-gateway-api-1    integration-gateway-gateway-api    "python app.py"                       gateway-api 22 hours ago    Up 2 hours (healthy)    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
integration-gateway-internal-api-mock-1 integration-gateway-internal-api-mock "python app.py"                       internal-api-mock 22 hours ago    Up 2 hours (healthy)
integration-gateway-postgres-1       postgres:15-alpine                "docker-entrypoint.s..."             postgres    22 hours ago    Up 2 hours (healthy)    5432/tcp
integration-gateway-transformer-1     integration-gateway-transformer    "python app.py"                       transformer 22 hours ago    Up 2 hours (healthy)
```

### 9.2 Verificar healthchecks

```
curl http://localhost:8000/health # gateway-api: healthy
curl http://localhost:8300/health # audit-api: healthy
```

*Evidência 02: Endpoints /health respondendo corretamente*

```
{"hostname": "047b5d55df9a", "service": "gateway-api", "status": "healthy", "transformer_url": "http://transformer:8100/transform"}
{"hostname": "d6f0a6f87593", "service": "audit-api", "status": "healthy"}
→ integration-gateway git:(develop) █
```

### 9.3 Payload válido: resposta esperada 201

```
sh scripts/send_valid_order.sh
# transformer_version: stable-v1, status: success
```

*Evidência 03: Resposta 201 com status success*

```
{"correlation_id": "demo-valid-001", "gateway": "accepted", "transformer_response": {"correlation_id": "demo-valid-001", "internal_response": {"internal_hostname": "4b2c2de35e9f", "legacy_order_id": "EXT-2026-0001", "legacy_protocol": "LEGACY_ORDER_V1", "processed_at": "1778079145.124516", "status": "accepted", "status": "success", "transformer_version": "stable-v1"}, "transformer_status": "201"}
→ integration-gateway git:(develop) █
```

### 9.4 Payload inválido: resposta esperada 400

```
sh scripts/send_invalid_order.sh
# error: missing required external fields
```

*Evidência 04: Resposta 400 com campos ausentes*

```
{"correlation_id": "demo-invalid-001", "gateway": "accepted", "transformer_response": {"correlation_id": "demo-invalid-001", "error": "missing required external fields: ['order_id']", "status": "failed", "transformer_version": "stable-v1"}, "transformer_status": "400"}
→ integration-gateway git:(develop) █
```

### 9.5 Consultar auditoria

```
curl http://localhost:8300/audits/summary
# SUCCESS: 3 | stable-v1
```

*Evidência 05: /audits/summary no fluxo normal*

```
{
  "correlation_id": "demo-valid-001",
  "gateway": "accepted",
  "transformer_response": {
    "correlation_id": "demo-valid-001",
    "internal_response": {
      "internal_hostname": "898595578de2",
      "legacy_order_id": "EXT-2026-0001",
      "legacy_protocol": "LEGACY_ORDER_V1",
      "processed_at": 1778081628.4963708,
      "status": "accepted",
      "status": "success",
      "transformer_version": "stable-v1",
      "transformer_status": 201
    }
  }
}

{
  "correlation_id": "demo-invalid-001",
  "gateway": "accepted",
  "transformer_response": {
    "correlation_id": "demo-invalid-001",
    "error": "missing required external fields: ['order_id']",
    "status": "failed",
    "transformer_version": "stable-v1",
    "transformer_status": 400
  }
}

[{"status": "FAILED", "total": 1, "transformer_version": "stable-v1"}, {"status": "SUCCESS", "total": 1, "transformer_version": "stable-v1"}]
```

➤ integration-gateway git: (develop) []

## 10. Incidente: transformer broken-v2

Uma nova versão do transformer foi publicada com bug de contrato. O broken-v2 envia campos com nomes incorretos ao sistema legado, causando falha na integração.

### 10.1 Divergência de contrato

| Versão    | Campos enviados           | Campos esperados pelo legado                          |
|-----------|---------------------------|-------------------------------------------------------|
| broken-v2 | order_id, items, sku, qty | legacy_order_id, legacy_items, legacy_sku, legacy_qty |

### 10.2 Reproduzir o incidente

```
docker compose -f docker-compose.yml -f docker-compose.incident.yml up -d --build transformer
sh scripts/send_valid_order.sh
# 502: transformer_version: broken-v2, status: failed
```

*Evidência 07: Resposta 502 retornada pelo broken-v2*

```
aguardando transformer healthy...
aguardando transformer healthy...
aguardando transformer healthy...
aguardando transformer healthy...
aguardando transformer healthy...
aguardando transformer healthy...
aguardando transformer healthy...
aguardando transformer healthy...
aguardando transformer healthy...
aguardando transformer healthy...
{"correlation_id":"incident-1778880854","gateway":"accepted","transformer_response":{"correlation_id":"incident-1778880854","internal_response":{"error":"contract violation","missing_fields":["legacy_order_id","legacy_customer_code","legacy_total_items","legacy_items"],"status":"rejected"},"internal_status":422,"status":"failed","transformer_version":"broken-v2"},"transformer_status":502}
integration-gateway git:(develop) █
```

## 11. Diagnóstico pelo Log

O diagnóstico foi feito cruzando dois logs. O internal-api-mock registrou os campos ausentes no payload recebido. O transformer registrou o status HTTP 422 retornado pelo mock.

### 11.1 Log do transformer

```
[transformer] starting on port 8100 version=broken-v2
[transformer] FAILED correlation_id=demo-valid-001 internal_status=422
```

*Evidência 08: Logs do transformer broken-v2*

```
transformer-1 | [transformer] starting on port 8100 version=broken-v2
transformer-1 | * Serving Flask app 'app'
transformer-1 | * Debug mode: off
transformer-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
transformer-1 | * Running on all addresses (0.0.0.0)
transformer-1 | * Running on http://127.0.0.1:8100
transformer-1 | * Running on http://172.23.0.4:8100
transformer-1 | Press CTRL+C to quit
transformer-1 | 127.0.0.1 - - [06/May/2026 15:09:04] "GET /health HTTP/1.1" 200 -
transformer-1 | [transformer] FAILED correlation_id=incident-1778880144 internal_status=422
transformer-1 | 172.22.0.4 - - [06/May/2026 15:09:04] "POST /transform HTTP/1.1" 502 -
transformer-1 | 127.0.0.1 - - [06/May/2026 15:09:14] "GET /health HTTP/1.1" 200 -
transformer-1 | 127.0.0.1 - - [06/May/2026 15:09:25] "GET /health HTTP/1.1" 200 -
transformer-1 | 127.0.0.1 - - [06/May/2026 15:09:35] "GET /health HTTP/1.1" 200 -
transformer-1 | 127.0.0.1 - - [06/May/2026 15:09:46] "GET /health HTTP/1.1" 200 -
transformer-1 | 127.0.0.1 - - [06/May/2026 15:09:56] "GET /health HTTP/1.1" 200 -
```

### 11.2 Log do internal-api-mock

```
[internal-api-mock] rejected
missing=[legacy_order_id, legacy_customer_code,
        legacy_total_items, legacy_items]
payload={...}
```

Evidência 09: Logs do internal-api-mock rejeitando campos

```
internal-api-mock-1 | 127.0.0.1 - - [06/May/2026 15:08:53] "GET /health HTTP/1.1" 200 -
internal-api-mock-1 | 127.0.0.1 - - [06/May/2026 15:09:04] "GET /health HTTP/1.1" 200 -
internal-api-mock-1 | [internal-api-mock] rejected missing=[legacy_order_id, legacy_customer_code, legacy_total_items, legacy_items] payload={'order_id': 'EXT-2026-0001', 'customer_code': 'CUST-ALPHA', 'total_items': 30, 'items': [{'sku': 'MB-001', 'qty': 10}, {'sku': 'RAM-008', 'qty': 20}], 'source_contract': 'BROKEN_EXTERNAL_ORDER_V3', 'transformed_at': 1778080144.83715, 'transformer_hostname': '3aea3f03ad36'}
internal-api-mock-1 | 127.22.0.3 - - [06/May/2026 15:09:04] "POST /Legacy/orders HTTP/1.1" 422 -
internal-api-mock-1 | 127.0.0.1 - - [06/May/2026 15:09:15] "GET /health HTTP/1.1" 200 -
```

### 11.3 Confirmação via auditoria

```
curl http://localhost:8300/audits/summary
# FAILED | broken-v2: 1 registro
```

Evidência 10: /audits/summary durante o incidente

```
[{"status": "FAILED", "total": 14, "transformer_version": "broken-v2"}, {"status": "SUCCESS", "total": 5, "transformer_version": "stable-v1"}, {"status": "FAILED", "total": 2, "transformer_version": "stable-v1"}]
integration-gateway git:(develop) []
```

Causa raiz: campos renomeados sem atualizar o contrato legado, violando o contrato `LEGACY_ORDER_V1`.



## 12. Rollback e Correção

O rollback consistiu em recompilar o transformer sem o override do arquivo de incidente, restaurando a versão stable-v1.

### 12.1 Restaurar versão estável

```
docker compose up -d --build transformer
# versão stable-v1 volta ao ar
```

Evidência 11: Rollback para stable-v1

```
=> [internal-api-mock] resolving provenance for metadata file 0.0s
=> [transformer] resolving provenance for metadata file 0.0s
[+] up 5/5
✓ Image integration-gateway-transformer Built 4.1s
✓ Image integration-gateway-internal-api-mock Built 4.1s
✓ Container integration-gateway-postgres-1 Healthy 21.0s
✓ Container integration-gateway-internal-api-mock-1 Healthy 32.5s
✓ Container integration-gateway-transformer-1 Started 22.4s
• integration-gateway git:(develop) [ ]
```

### 12.2 Validar

```
docker compose logs transformer | grep version=stable-v1
sh scripts/send_valid_order.sh
# 201: transformer_version: stable-v1, status: success

curl http://localhost:8300/audits/summary
# SUCCESS | stable-v1: 4 registros
# FAILED | broken-v2: 1 registro (histórico imutável)
```

Evidência 12: Validação pós-rollback

```
{
  "correlation_id": "rollback-1778080854",
  "gateway": "accepted",
  "transformer_response": {
    "correlation_id": "rollback-1778080854",
    "internal_response": {
      "internal_hostname": "da8abecfbd8",
      "legacy_order_id": "EXT-2026-0001",
      "legacy_protocol": "LEGACY_ORDER_V1",
      "processed_at": "1778080854.5110328",
      "status": "accepted",
      "status": "success",
      "transformer_version": "stable-v1",
      "transformer_status": "201"
    }
  }
}

[{"status": "FAILED", "total": 14, "transformer_version": "broken-v2"}, {"status": "SUCCESS", "total": 6, "transformer_version": "stable-v1"}, {"status": "FAILED", "total": 2, "transformer_version": "stable-v1"}]
• integration-gateway git:(develop) [ ]
```

Os registros de falha não foram corrigidos. A auditoria é imutável e serve como prova histórica do ocorrido.

## 13. Como a Auditoria Ajudou

O endpoint `/audits/summary` agrupou registros por status e `transformer_version`, tornando o diagnóstico imediato.

| status  | transformer_version | total |
|---------|---------------------|-------|
| SUCCESS | stable-v1           | 4     |
| FAILED  | broken-v2           | 1     |
| FAILED  | stable-v1           | 1     |

- Quando os erros começaram: versão broken-v2
- Quando foram resolvidos: versão stable-v1 voltou ao ar
- O que falhou: contrato legado violado

- docker inspect confirma limites e hardening aplicados corretamente

## 14. Respostas às Perguntas do Projeto

### 14.1 Por que o Postgres não deve estar na rede externa?

O banco armazena registros de auditoria com dados sensíveis das integrações. Expô-lo à rede externa criaria um vetor de ataque direto ao dado persistido. O isolamento via `data_net` garante que somente os serviços autorizados (`transformer` e `audit-api`) consigam acessar o banco, em conformidade com o princípio do menor privilégio.

### 14.2 Por que o Gateway precisa de duas redes?

O gateway-api é o único ponto de contato entre o mundo externo e a infraestrutura interna. Para cumprir esse papel ele precisa estar em `external_net` (receber requisições externas) e em `internal_net` (encaminhá-las ao `transformer`). Sem participar das duas redes, o serviço não conseguiria fazer a ponte entre os domínios.

### 14.3 Qual serviço conhece o contrato legado?

O `transformer` é o único serviço que conhece o contrato `LEGACY_ORDER_V1`. Ele traduz os campos externos (`order_id`, `customer_code`, `total_items`, `items`) para os campos esperados pelo sistema legado (`legacy_order_id`, `legacy_customer_code`, etc.). Qualquer mudança de contrato deve ser implementada e testada exclusivamente nele antes do deploy.

### 14.4 Como a falha foi identificada pelos logs?

O diagnóstico foi feito cruzando dois logs: o `internal-api-mock` registrou exatamente os campos ausentes no payload, indicando o que o `transformer` deveria ter enviado. O `transformer`, por sua vez, registrou o status HTTP 422 retornado pelo mock. Com o campo `transformer_version` presente em cada registro de auditoria, foi possível correlacionar imediatamente as falhas à versão `broken-v2`.

### 14.5 O rollback corrigiu as integrações já registradas como falha?

Não. Os registros de falha gerados pela versão `broken-v2` permanecem inalterados na base de auditoria. Isso é intencional: a auditoria serve como prova histórica imutável do que aconteceu, permitindo rastrear quando os erros começaram e terminaram. O rollback corrige apenas as integrações processadas a partir do momento em que o `stable-v1` voltou ao ar.

### 14.6 Qual a importância de versionar contratos?

A presença do campo `transformer_version` em cada registro de auditoria foi decisiva para o diagnóstico. Sem esse campo, seria impossível determinar se as falhas foram causadas por uma mudança de versão do `transformer`, por dados inválidos dos clientes ou por instabilidade do sistema legado. O versionamento transforma a auditoria de um simples log de erros em uma ferramenta de rastreabilidade precisa.

## 15. Aprendizados Principais

- Isolamento de redes por responsabilidade evita exposição desnecessária de serviços
- `depends_on` com `healthcheck` garante inicialização segura e ordenada dos serviços
- Auditoria com `transformer_version` é fundamental para rastrear incidentes com precisão
- Hardening (`read_only`, `cap_drop`, `no-new-privileges`) é configurável no Compose sem alterar o código
- Rollback rápido é possível pois o `docker-compose.incident.yml` sobrescreve apenas o build do transformer
- Logs estruturados (`[serviço] campo=valor`) permitem diagnóstico sem acesso direto ao banco